

ZODA — Guía del Código

Qué hace cada parte y por qué

SCRIPT 1 ZODA_03_06_2026.py — El motor de cálculo

Este script es el cerebro del sistema. Lo ejecutas una vez al día y produce todos los Z-scores, múltiplos y baskets que luego el dashboard visualiza. Nunca lo toques: contiene la lógica de negocio original del notebook 2020.

Sección 1 · Rutas de archivos

```
RUTA_BASE = r"C:\Users\usuario\...\ZODA\shortcut"
RUTA_ZODA = r".. \ZODA.xlsm"
RUTA_MULTIPLOS_CSV = os.path.join(RUTA_BASE, "Multiplos_diarios.csv")
```

Qué hace: Define las direcciones de todos los archivos que el script necesita leer o escribir. El prefijo `r` antes de las comillas indica que la barra invertida `\` es un carácter literal (no un comando especial).

Por qué: Si en algún momento los archivos cambian de carpeta, solo hay que editar estas líneas — el resto del script no cambia.

Sección 2 · Parámetros globales

```
ANIOS_Z = [1, 3, 5, 10]
PESOS_Z = [0.33, 0.33, 0.33, 0.00]

PESOS_FINAN = [0.33, 0, 0.33, 0, 0.33, 0, 0, 0] # Bancos
PESOS_EX_FINAN = [0.25, 0, 0.25, 0, 0.25, 0, 0.25, 0] # Resto
```

Qué hace: Define los pesos que gobiernan cómo se construye el Z-score compuesto. Son los únicos parámetros que reflejan criterio de inversión directamente en el código.

Por qué cada decisión:

- **PESOS_Z = [0.33, 0.33, 0.33, 0.00]:** el Z compuesto promedia tres ventanas temporales con igual peso. La de 10 años tiene 0 porque en mercados latinoamericanos esa historia incluye crisis que no son representativas del régimen actual.
- **PESOS_FINAN (sin EV/EBITDA):** para bancos, el EBITDA no tiene significado económico — el financiamiento es parte del negocio, no un costo a eliminar. Por eso su peso es 0.
- **PESOS_EX_FINAN (4 múltiplos iguales):** para empresas industriales, consumo y minería los cuatro múltiplos capturan ángulos distintos: ganancias (PE), ventas (PS), valor en libros (PB) y generación de caja (EV/EBITDA).

Pregunta frecuente: ¿Por qué solo trailing y no forward? Los pesos son 0 para todos los múltiplos forward. La razón: las estimaciones forward de CIQ para empresas latinoamericanas tienen cobertura limitada y baja calidad. El trailing con datos reales es más confiable.

Sección 3 · Lectura del Excel

```
def leer_hoja_excel(ruta, hoja, n_filas=7600):
    df = pd.read_excel(ruta, sheet_name=hoja,
                      header=None, nrows=n_filas,
                      engine="openpyxl", dtype=object)
    df.columns = [f"column_{i}" for i in range(len(df.columns))]
    return df
```

Qué hace: Lee una hoja de Excel y la convierte en una tabla Python (DataFrame). El parámetro `dtype=object` le dice a Python que lea cada celda tal como está, sin intentar adivinar si es número, fecha o texto.

Por qué `dtype=object`: Sin este parámetro, pandas puede malinterpretar celdas con errores de Bloomberg como #N/A y convertirlas en tipos incompatibles que luego rompen los cálculos. Con `dtype=object` todo llega como texto plano y el código decide después qué hacer con cada celda.

Sección 4 · Actualización de posiciones DQ_IN

```
def actualizar_DQ_IN(df_quest, dq_in_bd):
    # Busca la última fecha ya registrada en ZODA
    ultima_fecha = pd.to_datetime(dq_in_bd["Fecha"]).max()
    # Toma del QUEST solo las filas más nuevas
    nuevas = df_quest[df_quest["Fecha"] > ultima_fecha]
    return pd.concat([dq_in_bd, nuevas], ignore_index=True)
```

Qué hace: Actualiza la base de posiciones del fondo (hoja DQ_IN de ZODA.xlsm) con los datos más recientes del QUEST_BD.xlsm que viene de la SBS. Solo agrega las filas nuevas — no reescribe toda la historia.

Por qué incremental: Preservar la historia completa de posiciones es crítico para el bubble chart del dashboard, que puede mostrar cualquier fecha pasada. Si se reescribiera toda la hoja en cada ejecución, se perdería el historial.

Sección 7 · Limpieza de datos (LimpiandoCIQ y LimpiandoBBG)

LimpiandoCIQ — datos trimestrales de Capital IQ

```
_COLS_ALL = ["IQ_NI", "IQ_TOTAL_REV", "IQ_TOTAL_EQUITY", "IQ_EBITDA",
             "Px_last", "CUR_MKT_CAP", "CURR_ENTP_VAL"]

data_ciq = data_emp.iloc[:, :4].dropna(axis=0, how="all")
data_ciq = forwards_one_year(data_ciq, ["Fwd_IQ_NI", ...])
```

Qué hace: Extrae Utilidad Neta, Ventas, Patrimonio y EBITDA de la hoja correspondiente, elimina filas completamente vacías, y agrega los valores forward (proyectados a 12 meses) para calcular los múltiplos Fwd.

Nombres por posición fija: Las columnas del Excel contienen fórmulas como `"IQ_NI"` que Python no puede evaluar — devuelve vacío. La solución es asignar nombres por posición (columna 0 = `IQ_NI`, columna 1 = `IQ_TOTAL_REV`, etc.) en lugar de leer el encabezado. Esto hace el código inmune a cambios en los encabezados del Excel.

LimpiandoBBG — datos diarios de Bloomberg

```
data_bbg = data_emp_.iloc[:, 4:] # ← SIN dropna()
data_bbg = data_bbg.apply(pd.to_numeric, errors="coerce").fillna(0)
```

Qué hace: Extrae precio, Market Cap y Enterprise Value de Bloomberg. El `SIN dropna()` es la corrección más importante del código.

Por qué no hay dropna: Bloomberg rellena con `#N/A` `N/A` los días anteriores al IPO de una empresa. Python los convierte en vacíos (NaN). Si se eliminaran esas filas, las fechas quedarían desalineadas: los datos de AUNA (que cotiza desde 2022) recibirían fechas de 2008. Sin `dropna`, las filas vacías permanecen y el relleno con 0 las trata como 'sin dato', que luego el cálculo de múltiplos excluye correctamente.

Sección 8 · Cálculo de múltiplos

```
def _ciq_diario(col):
    s = ciq[col].copy()
    s.index = s.index.to_period("Q") # trimestral → período Q
    return Series(s.reindex(fechas_q).ffill().values, index=fechas_nuevas)

def _ratio(num, den, nombre):
    num_clean = num.replace(0, np.nan) # 0 BBG = sin dato
    den_clean = den.replace(0, np.nan) # 0 CIQ = sin dato
    return num_clean / den_clean
```

Qué hace: Divide el numerador diario (precio de Bloomberg) entre el denominador trimestral (fundamentales de CIQ) para obtener el múltiplo diario. El truco es hacer que el dato trimestral 'se repita' para todos los días de ese trimestre.

El problema del desfase trimestral: Bloomberg actualiza el precio cada día. CIQ actualiza Utilidad Neta solo al cierre de cada trimestre. Para calcular P/E diario se necesita saber qué utilidad corresponde a cada día: la respuesta es la del último trimestre cerrado. La función `ffill()` ('forward fill') propaga el dato del trimestre hacia adelante hasta que llegue el siguiente reporte.

Por qué se convierte 0 a NaN: Un Market Cap de 0 significa 'Bloomberg no tiene dato ese día' (empresa sin cotización aún). Si se dividiera 0 / utilidad = 0, el múltiplo P/E aparecería como cero — un número falso que contaminaría el Z-score. Convertirlo a NaN hace que la operación devuelva 'sin dato', que es la respuesta correcta.

Sección 9 · Cálculo del Z-score

Función Z() — rolling Z-score

```
def Z(prueba_z, anios):
    window = anios * 260          # 260 días hábiles por año
    media = prueba_z.rolling(window=window, min_periods=window).mean()
    std = prueba_z.rolling(window=window, min_periods=window).std()
    return (prueba_z - media) / std
```

Qué hace: Para cada día, toma los últimos N días hábiles, calcula la media y desviación estándar de ese período, y estandariza el valor actual. El resultado indica cuántas desviaciones estándar se aleja hoy del promedio de los últimos N años.

Por qué min_periods=window: Con este parámetro, los primeros N días (donde aún no hay ventana completa) devuelven NaN en lugar de un Z parcial. Evita señales espurias al inicio de la historia.

Función Creacion_Z_Final() — Z compuesto

```
for i, anios in enumerate(ANIOS_Z):          # [1, 3, 5, 10]
    z_i = Z(prueba_z, anios) * PESOS_Z[i]    # Z_N × peso
    z_acum = z_acum.add(z_i, fill_value=0)    # suma acumulada

# Ponderar por múltiplo y colapsar a una sola serie
pesos_mult = PESOS_FINAN if es_finan else PESOS_EX_FINAN
z_final = (z_acum * pesos_mult).sum(axis=1)
```

Qué hace: Combina 8 múltiplos × 4 ventanas temporales en un único número por día: el Z-score compuesto. El proceso tiene dos niveles de ponderación:

- **Nivel 1 — por ventana temporal:** $Z_{1año} \times 0.33 + Z_{3años} \times 0.33 + Z_{5años} \times 0.33 + Z_{10años} \times 0$.
- **Nivel 2 — por múltiplo:** cada múltiplo (PE, PS, PB, EV/EBITDA) recibe su peso según el sector.

Por qué fill_value=0: Algunos múltiplos pueden no tener dato en ciertas fechas (empresa sin EV/EBITDA disponible, por ejemplo). El fill_value=0 hace que esa ausencia contribuya con 0 a la suma en lugar de anular todo el cálculo con NaN.

Resultado final: Un número por empresa por día. Ejemplo: BAP cierra con $Z = +1.76$ hoy. Eso significa que su múltiplo compuesto está 1.76 desviaciones estándar por encima de su media histórica — señal de valorización elevada.

Sección 10 · Cálculo de baskets

```
mc = market_cap[mc_cols]
pesos = mc.div(mc.sum(axis=1), axis=0) # peso = MC_empresa / MC_total
df_basket[mult] = pesos.multiply(dfs[mult]).sum(axis=1)
```

Qué hace: Para cada empresa en cartera, calcula el múltiplo promedio de sus comparables globales ponderado por Market Cap. Si AB InBev es 10 veces más grande que el resto del basket de Backus, el múltiplo del basket refleja principalmente la valorización de AB InBev.

Por qué ponderación por Market Cap: Es el estándar de la industria (J.P. Morgan, Goldman Sachs, Barclays). La alternativa sería simple promedio, pero eso daría el mismo peso a una empresa de USD 1,000M que a una de USD 100,000M. El Market Cap refleja el verdadero peso económico de cada comparable en su sector.

Resultado: El Z_Basket de BAP = Z del ratio [múltiplo BAP / múltiplo basket]. Si BAP siempre cotizó con un 10% de prima sobre sus pares y hoy cotiza con 30%, el Z_Basket será positivo — señal de prima inusualmente alta.

Sección 14b · Exportar Multiplos_diarios.csv

```
for ticker, df_mult in matriz_multiplos.items():
    df_tmp.insert(1, "Ticker", ticker)    # añade columna Ticker
    partes.append(df_tmp)

csv_final = pd.concat(partes, ignore_index=True)
csv_final.to_csv(ruta_csv, index=False, encoding="utf-8-sig")
```

Qué hace: Al terminar de calcular todos los múltiplos, los apila en un único CSV en formato largo: una fila por empresa por día. Este archivo es el puente con el dashboard — lo que antes tardaba 55 segundos de recalcular ahora tarda 1 segundo de leer.

Formato largo vs ancho: En lugar de tener columnas como 'BAP_Trail_PE', 'IFS_Trail_PE', etc. (formato ancho), se usa una columna Ticker y una columna Trail_PE para todas las empresas (formato largo). Es más compacto y más fácil de filtrar por empresa o por múltiplo.

encoding utf-8-sig: El BOM (Byte Order Mark) al inicio del archivo garantiza que Excel lo abra correctamente con acentos y ñ — sin él, caracteres como 'Utilidad' pueden mostrarse como 'Utilidad'.

SCRIPT 2 zoda_dashboard_v4.py — El generador del dashboard

Este script lee los datos ya calculados y genera el HTML interactivo. No hace cálculos de inversión — eso lo hizo el Script 1. Su trabajo es presentar los datos de forma visual y usable.

Función cargar_bd() — cargar los Z-scores

```
df = pd.read_excel(RUTA_ZODA, sheet_name="BD", header=0)
df[z_cols] = df[z_cols].replace(0, np.nan)
```

Qué hace: Lee la hoja BD del ZODA.xlsm — la tabla de 86 columnas Z-score × días que el Script 1 actualiza. Reemplaza los ceros por 'sin dato' (NaN).

Por qué replace(0, np.nan): En BD, un cero significa 'ZODA aún no tenía datos para esa empresa en esa fecha' (por ejemplo, AUNA antes de su IPO). Si se dejara como cero, los gráficos mostrarían una línea plana en 0 en lugar de cortar la serie en la fecha correcta.

Función cargar_exposiciones() — tamaño de burbujas

```
pxq = df[df["Dato"] == "PxQ"] # solo filas PxQ
for sbs in sbs_cols:
    row[sbs] = pxq[sbs].sum() / 100_000 # suma Fondo1+2+3
```

Qué hace: Para cada fecha disponible en el DQ_IN, suma la exposición de los tres fondos (Fondo 1, 2 y 3) para cada código SBS. El resultado es la posición total de AFP Integra en cada activo en millones de soles — el tamaño de cada burbuja en el Chart 02.

Por qué solo filas PxQ: El DQ_IN tiene varios tipos de filas: cantidades (Q), precios (Px), precio×cantidad (PxQ) y otras. El valor económico de la posición es PxQ — el monto total invertido. Es el único número comparable entre acciones con precios distintos.

Función build_bubble_data() — datos del bubble chart

```
def _safe_z(val):
    try: return float(val) # fuerza conversión a número
    except: return np.nan

row = {col: row_series[col] for col in bd.columns} # dict Python puro
z_abs = _safe_z(row.get(z_col, np.nan))
```

Qué hace: Construye el dataset del bubble chart: para cada fecha y cada empresa, recoge el Z absoluto (eje X), el Z vs Basket (eje Y) y la exposición en S/. millones (tamaño).

Por qué _safe_z y el dict Python: Este fue el bug más difícil de diagnosticar: cuando se extrae un valor de una fila de pandas sin esta conversión, el objeto lleva metadatos internos de fecha que Plotly interpreta como un timestamp. El eje X mostraba '18:59:59 Dec 31, 1969' en lugar del Z-score. Convertir a dict Python puro y luego a float elimina cualquier metadato y garantiza que Plotly recibe un número limpio.

Función `_z_por_ventana_desde_multiplos()` — pesos dinámicos Chart 1

```
for anios in [1, 3, 5, 10]:
    window = anios * 26          # 26 puntos/año en datos bi-semanales
    z = ((serie - media) / std) * peso_multiplo
    resultado[f"{anios}Y"] = z   # guarda cada ventana por separado
```

Qué hace: Para cada empresa, precalcula el Z de cada ventana temporal (1Y, 3Y, 5Y) a partir de los múltiplos reales del CSV. Estos cuatro vectores se guardan en el HTML como datos separados.

Por qué es necesario: El panel de pesos del Chart 1 permite al usuario ajustar en tiempo real cuánto peso dar a cada ventana. Para que el gráfico responda sin reconectar Python, el HTML debe tener los cuatro vectores disponibles. JavaScript los combina en el cliente: `z_final = p1×Z_1Y + p3×Z_3Y + p5×Z_5Y + p10×Z_10Y`.

Por qué 26 puntos/año y no 260: El CSV de múltiplos se guarda con resolución bi-semanal (un dato cada dos semanas) para mantener el HTML en ~13 MB en lugar de 100+ MB. Con bi-semanal hay ~26 puntos por año en lugar de 260 (diario). La lógica del Z es idéntica, solo cambia la granularidad.

Función `generar_html()` — el HTML final

```
html = HTML_TEMPLATE
html = html.replace("__HIST_DATA__", json.dumps(hist_traces))
html = html.replace("__BUBBLE_DATA__", json.dumps(bubble_data))
html = html.replace("__LOGO_B64__", logo_b64)
```

Qué hace: Toma la plantilla HTML (con marcadores como `__HIST_DATA__`) y los reemplaza uno a uno con los datos reales en formato JSON. El resultado es un único archivo HTML donde los datos están literalmente escritos dentro del código JavaScript.

Por qué incrustar los datos en el HTML: La alternativa sería que el HTML llamara a un servidor Python cada vez que el usuario hace click. Al incrustar todo, el HTML es completamente autónomo — funciona sin red, sin Python activo, se puede enviar por correo y abrirse en cualquier computadora. El costo es el tamaño del archivo (~13 MB), que es aceptable para uso interno.

Por qué `json.dumps`: JSON es el lenguaje que Python y JavaScript comparten para intercambiar datos. `json.dumps` convierte una tabla de Python en texto que JavaScript puede leer directamente — sin conversor adicional.